

Serverless Functions

A short primer: what a function is, why a model call lives in one, and what cold, warm and the time limit mean

Written for the *put the model behind a URL* lab; the ideas apply to every function in the course. The platform in the course is AWS Lambda; the ideas are the same on every provider. Names and limits are September 2026 and will change; the mechanism will not.

1. What it is

A **function** is code you upload, with no server to run it on. The platform keeps the code and, when a request arrives, starts a small isolated environment (a **sandbox**), runs your handler once for that request, and keeps the sandbox around for a while in case another request comes. You choose the memory size and a maximum running time. You are billed per request and per millisecond of running time at that memory size. When nothing is calling the function, nothing runs and nothing is billed.

The handler is an ordinary function: an event in, a response out. Everything else, the machine, the operating system, the runtime, scaling, is the platform's.

2. How one request runs

1. A request arrives (over a URL, from a queue, on a schedule).
2. If a warm sandbox is free, the platform hands it the request. If not, it creates one: allocates the sandbox, loads the runtime, imports your code. This is the **cold start**.
3. Your handler runs. For the triage function that is one model call.
4. The response goes back. The sandbox stays alive, idle, for minutes; if another request arrives it is **warm** and only the handler runs.
5. After enough idleness the sandbox is discarded. The next request is cold again.

A burst of simultaneous requests creates many sandboxes at once, each one cold. That is what "scales per request" means, and what it costs.

3. Cold and warm

	COLD	WARM
what happens before the handler	create the sandbox, load the runtime, import your code	nothing
what dominates	importing dependencies: for the triage function, the model's SDK, about 1.5 s	the handler itself
when it happens	the first request into a new sandbox; every sandbox in a burst; after minutes idle; after any configuration change	any later request into a live sandbox
what you can do about it	smaller packages, fewer imports, more memory (which is also more CPU), or paying to keep sandboxes warm	nothing needed

Measured in the *put the model behind a URL* lab, for the triage function at 512 MB: a cold round trip of about 3.4 s, of which 1.5 s is the import, 1.3 s the first model call, and 0.5 s the network and the runtime; warm round trips of about 0.9 s, of which the model is 0.7 s.

The import happens before the handler, so it does not count against the time limit below, but the caller waits for it.

4. The time limit

Every invocation has a maximum running time, set per function, from one second to fifteen minutes. When it is reached, the sandbox is killed where it stands: no cleanup, no partial result, and the caller receives an error. The model may well have answered into the void.

For a model call this is the important property. One call takes seconds; a limit set below its latency kills every call; a chain of calls will cross any sensible limit eventually. Anything that does more than one call per request has to keep its state somewhere that survives the death of the process running it, and record each step as it lands, which is why the recorder in the lab writes one line per call.

The right limit is a number you measure, not a default: the warm 95th-percentile latency, rounded up, plus a margin for the tail you are willing to pay for.

5. Concurrency, quotas, and what "scaling" means here

The platform will run as many sandboxes as there are simultaneous requests, up to the account's **concurrency limit** (in the low thousands by default). There is no autoscaler to configure, and nothing to tune.

That is not the same as capacity for a model call. The model behind the function has its own limit, a **quota** in requests and tokens per minute on your key. Adding sandboxes past that quota adds waiting, not throughput: every sandbox waits on the same quota. A function scales the code around the model; it does not scale the model.

6. What it costs

Billing is requests $\times$ a small fixed price, plus running time $\times$ memory. A function that waits three seconds on a model pays for three seconds of waiting. Orders of magnitude, for the triage function at 512 MB:

	COST PER MILLION REQUESTS
a 200 ms handler that does not call a model	about \ \$2
a 3 s handler waiting on a model	about \ \$25
the same handler at 128 MB, since waiting needs no CPU	about \ \$6.50

For the course's support desk at a million tickets a month that is a few dollars, inside the free allowance. The cost changes character at high sustained throughput, where a machine that is always on becomes cheaper than paying for waiting by the millisecond. That crossover is a few requests a second for a handler that waits on a model, and tens for one that does not.

7. What a function is not

- It does not hold a connection open, so nothing can push to it; it is invoked.
- It keeps no state between requests. A warm sandbox's memory survives by accident, and you cannot rely on it.
- It cannot run past its limit.

A long-running worker, a queue consumer that never stops, a stateful loop, an open websocket: each of those lives elsewhere, and each arrives later in the course as its own placement decision.

Size limits, for reference. The uploaded package is capped at 50 MB zipped when uploaded directly and 250 MB unpacked; larger packages go through object storage or a container image, which may be up to 10 GB. Memory is chosen per function, from 128 MB to about 10 GB, and buys CPU in proportion. Scratch disk is a few hundred MB by default. The *put the model behind a URL* package is about 10 MB zipped and 38 MB unpacked, most of it the model's SDK and its dependencies; a model's own weights do not fit in a function's package beyond a few hundred megabytes, which is one reason *model serving* is its own topic later in the course.

8. The URL, and who may call it

A **function URL** is a public HTTPS address wired straight to one function, with no gateway in between. It can be anonymous or require callers to sign requests with AWS credentials. The Learner Lab requires the signature, which is why the *put the model behind a URL* lab's recorder and its `call.sh` sign every call; on an account you control, an anonymous URL with a shared secret in a header is the usual first step, and a proper gateway the second.

9. Words

TERM	MEANING
handler	the function the platform calls, once per request: event in, response out
sandbox	the isolated environment one invocation runs in; reused while warm
cold start	the first request into a new sandbox, including runtime load and your imports
warm	a request served by a sandbox that already started
time limit	the maximum running time per invocation; the sandbox is killed when it is reached
concurrency	how many sandboxes may run at once; an account limit
quota	the model vendor's limit per key on requests or tokens per minute; the ceiling a function cannot raise
package	what you upload: your code plus its dependencies, built for the platform's operating system, not your laptop's
function URL	a public address for one function, anonymous or signed